

ENTREGA 3 – ENTREGA CONTINUA (CD)
DevOps

Autor

Breitner Enrique Gonzalez Angarita

Prof.

Mario José Villamizar Cano

Universidad de los Andes

8 de mayo de 2026

Introducción

En este taller se implementa un pipeline completo de Entrega Continua (CD) usando AWS CodePipeline, CodeBuild, CodeDeploy y Amazon ECS con Fargate. El objetivo de esta actividad es extender el pipeline de Integración Continua (CI) del Taller 2 para incluir el despliegue automatizado con estrategia Blue-Green, garantizando que no hayan tiempos de inactividad en cada entrega.

La Entrega Continua representa la evolución natural de la Integración Continua: no solo validamos que el código compila y pasa las pruebas, sino que lo desplegamos automáticamente a producción de forma segura y reproducible. Con Blue-Green deployment, mantenemos dos ambientes idénticos y alternamos el tráfico entre ellos, permitiendo rollback instantáneo si algo falla.

Objetivos

- Agregar los archivos de configuración sobre el repositorio del microservicio para que el despliegue continuo funcione sobre AWS CodeDeploy.
- Configurar el Pipeline para que cada vez que se haga un commit en master se ejecute automáticamente.
- Implementar un pipeline de Entrega Continua completo con AWS CodePipeline
- Configurar despliegue automatizado en Amazon ECS Fargate con contenedores Docker
- Implementar estrategia Blue-Green deployment con AWS CodeDeploy para cero downtime
- Configurar Application Load Balancer con dos target groups para alternancia de tráfico
- Automatizar el flujo completo: GitHub push → Build → Test → Docker → ECR → Deploy Blue-Green.

Repositorio: https://github.com/breinergonza/devops_taller_2

Video la entrega: [Grabación_Entrega_Taller_3_Breitner_Gonzalez.mov](#)

Conceptos de Entrega Continua

La Entrega Continua (CD) automatiza el proceso completo desde un commit en el repositorio hasta la ejecución en producción. A diferencia de CI, donde solo validamos el código, CD se encarga de empaquetarlo, desplegarlo y verificar que funciona correctamente en el ambiente productivo.

Estrategia Blue-Green Deployment

Blue-Green deployment es una técnica que reduce el riesgo y el downtime al mantener dos ambientes de producción idénticos. En cualquier momento, solo uno de los ambientes (Blue) sirve tráfico real. El nuevo código se despliega en el ambiente inactivo (Green), se valida, y luego el tráfico se redirige del Blue al Green.

- Si hay algún problema, el rollback es instantáneo: simplemente se redirige el tráfico de vuelta al Blue.
- El ambiente Blue sirve el tráfico de producción actual
- El nuevo código se despliega en el ambiente Green sin afectar usuarios
- Se ejecutan validaciones de salud en Green antes de cambiar tráfico
- El ALB redirige el tráfico del Blue al Green de forma atómica
- Rollback instantáneo redirigiendo tráfico de vuelta al Blue en caso de fallo

AWS CodeDeploy con ECS

AWS CodeDeploy orquesta el proceso Blue-Green en ECS. Crea un nuevo TaskSet (Green) con la nueva versión de la imagen Docker, espera a que pase los health checks del ALB, y luego modifica los listeners del Load Balancer para redirigir el tráfico al nuevo TaskSet. El TaskSet anterior (Blue) se mantiene como respaldo durante un período configurable antes de ser eliminado.

Arquitectura del Pipeline

El pipeline implementado sigue un flujo de tres etapas orquestadas por AWS CodePipeline V2, disparado automáticamente por cada push a la rama main del repositorio en GitHub.

Arquitectura del Pipeline CI/CD - Taller 3

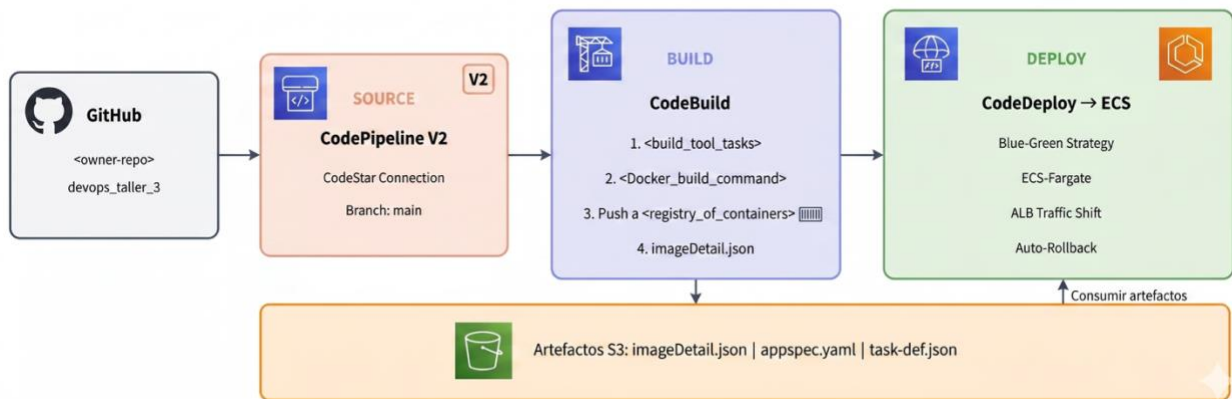


Ilustración 1. Arquitectura del pipeline CI/CD con las 3 etapas: Source, Build y Deploy

Flujo del Pipeline

El flujo completo del pipeline es el siguiente:

- **Source:** GitHub webhook detecta push en main → descarga código fuente como ZIP
- **Build:** CodeBuild ejecuta pruebas unitarias con pytest, construye imagen Docker, la sube a ECR y genera artefactos
- **Deploy:** CodeDeploy ejecuta Blue-Green deployment en ECS Fargate con el ALB

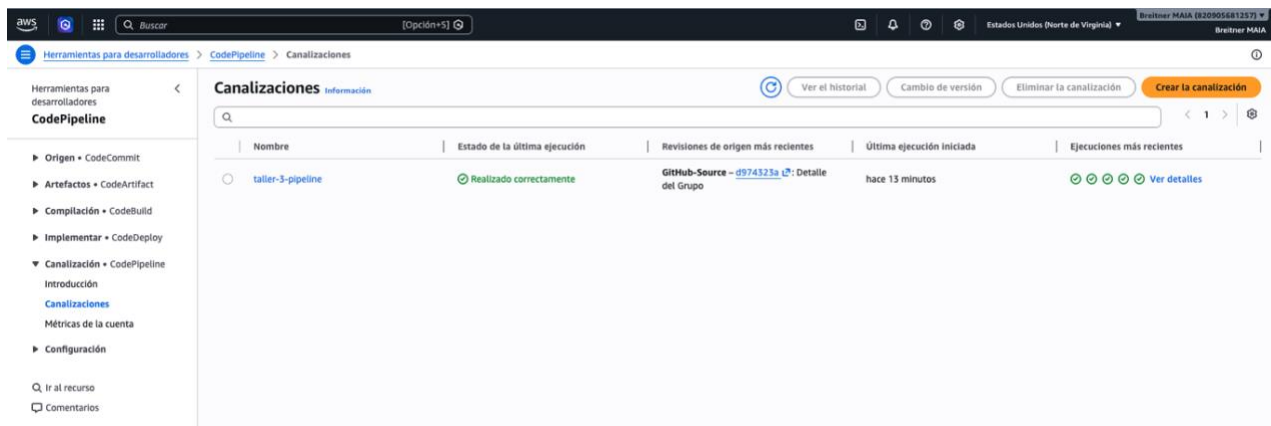


Ilustración 2. AWS CodePipeline Console - Vista del pipeline taller-3-pipeline.

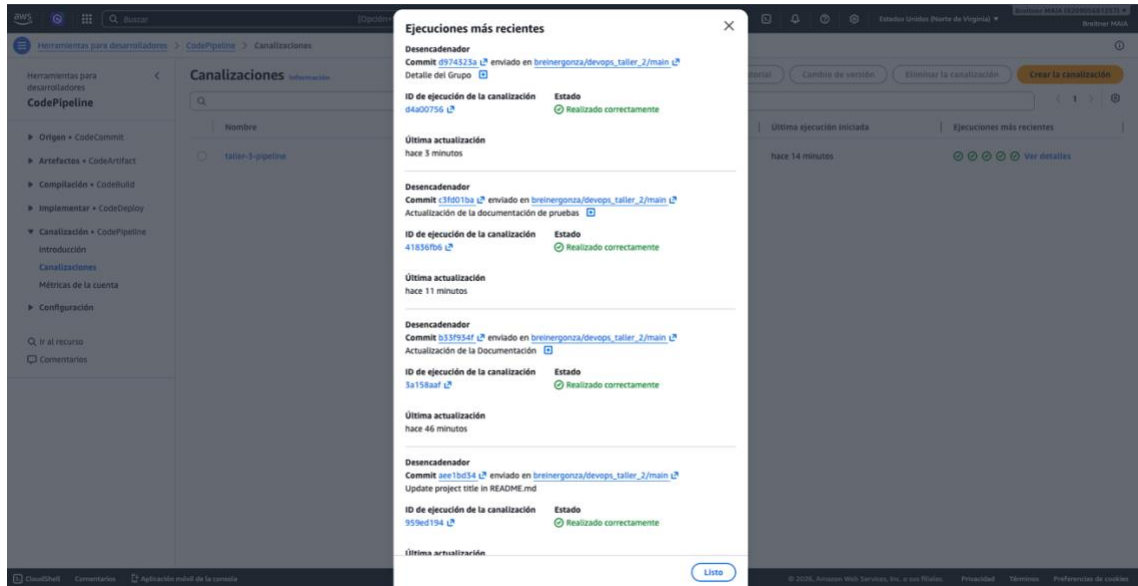


Ilustración 3. AWS CodePipeline Console - Ejecuciones del pipeline taller-3-pipeline.

Servicios AWS Utilizados

Servicio AWS	Función en el Pipeline
CodePipeline V2	Orquestación del pipeline con webhook GitHub
CodeBuild	Compilación, pruebas, construcción de imagen Docker
CodeDeploy	Despliegue Blue-Green en ECS
ECR	Registro de imágenes Docker
ECS Fargate	Ejecución de contenedores sin servidores
Application Load Balancer	Distribución de tráfico entre Blue y Green
Systems Manager	Almacenamiento seguro de secretos (SecureString)
CloudWatch Logs	Logs centralizados de contenedores y builds
S3	Almacenamiento de artefactos del pipeline

Configuración de AWS CodeBuild

AWS CodeBuild ejecuta la fase de Build del pipeline. Recibe el código fuente desde GitHub, ejecuta las pruebas unitarias, construye la imagen Docker del microservicio y la sube a Amazon ECR. La configuración se define en el archivo buildspec.yml.

	Ejecución del lote	Estado	Número de lote	Versión de origen	Remitente	Duración	Completado
☐	taller-2-ci:587a097d-e53d-4563-a979-593230974370	Realizado correctamente	23	cc3976396b512482aa4a0fa6e6a0fcca1ba0324	GitHub-Hookshot/03a48eb	3 minutos 9 segundos	hace 13 días
☐	taller-2-ci:ba97d667-3b70-4a1d-927e-7e37b79e5d48	Realizado correctamente	22	a55b1999d6a988519dad8ba90b3dbc44790ee324	GitHub-Hookshot/03a48eb	2 minutos 9 segundos	hace 13 días
☐	taller-2-ci:63101f04-2808-44af-b0af-0b4771467a23	Realizado correctamente	21	965b13c4132c5b87773800d46c8bbadb03eab38a	GitHub-Hookshot/03a48eb	2 minutos 9 segundos	hace 13 días
☐	taller-2-ci:32ce9e7-d9b3-4b64-0185-6aade56285a4	Realizado correctamente	20	c08009c7b4fa459683ef7504604441e55eece3f5	GitHub-Hookshot/03a48eb	2 minutos 9 segundos	hace 13 días
☐	taller-2-ci:ad8b30e0-bbf2-4cb4-bb5e-7eb0a79b7669	Realizado correctamente	11	fd899cfs2843cd02b5a56865cd336f79d5060e25	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 13 días
☐	taller-2-ci:07b28311-a626-46ab-bd9e-073d520558a2	Realizado correctamente	10	686a9a1ec967023ee311122e5d15fe75294951	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 13 días
☐	taller-2-ci:9544676f-0075-45ac-b559-e9fb6d115672	Realizado correctamente	9	27b73c36234857a2b7fb9ad9c74a2e482ab18c2	GitHub-Hookshot/03a48eb	2 minutos 5 segundos	hace 14 días

Ilustración 4. AWS CodeBuild - Build exitoso del proyecto taller-2-ci.

Archivo buildspec.yml

El buildspec.yml define las cuatro fases del proceso de construcción. Es el blueprint que le indica a CodeBuild exactamente qué hacer en cada paso del build.

- **Fase install:** Instala dependencias Python (pip install -r requirements.txt)
- **Fase pre_build:** Ejecuta pruebas unitarias con pytest, genera reportes JUnit y cobertura, autentica con ECR
- **Fase build:** Construye imagen Docker con tag del commit hash y la etiqueta latest
- **Fase post_build:** Sube imagen a ECR, genera imageDetail.json para CodeDeploy Blue-Green

Artefactos de Build

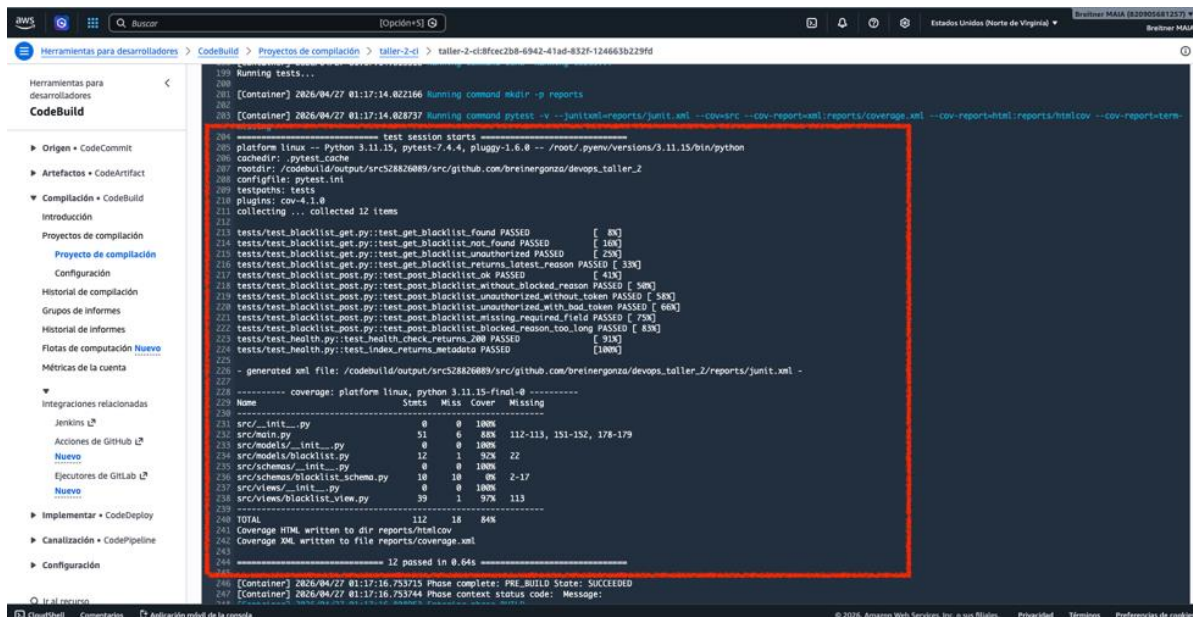
CodeBuild genera los siguientes artefactos que CodeDeploy necesita para ejecutar el despliegue Blue-Green:

- **imageDetail.json**: Contiene la URI de la imagen Docker recién construida en formato `{"ImageURI": "..."}`
- **appspec.yaml**: Especificación de CodeDeploy con configuración del servicio ECS
- **task-def.json**: Plantilla de Task Definition con placeholder `<IMAGE1_NAME>` para inyección de imagen.

Pruebas Automatizadas

Las pruebas se ejecutan automáticamente en la fase `pre_build` de CodeBuild antes de construir la imagen Docker. Si alguna prueba falla, el pipeline se detiene inmediatamente y no se realiza el despliegue, protegiendo el ambiente de producción.

Suite de Pruebas con pytest



The screenshot shows the AWS CodeBuild console interface. On the left, there's a sidebar with navigation options like 'Herramientas para desarrolladores', 'Artefactos', 'Compilación', and 'Implementar'. The main area displays the build logs for a project named 'taller-2-ci'. The logs show the execution of 'Running tests...' followed by 'test session starts'. A red box highlights the pytest output, which includes a list of tests and their results. The tests are all marked as 'PASSED'. Below the test results, there's a table showing coverage statistics for various files. The table has columns for 'Name', 'Sests', 'Miss', 'Cover', and 'Missing'. The 'TOTAL' row shows 112 tests, 18 misses, and 84% coverage. The logs conclude with '12 passed in 0.64s' and 'Phase complete: PRE_BUILD State: SUCCEEDED'.

```
199 Running tests...
200
201 [Container] 2026/04/27 01:17:14.022166 Running command mkdir -p reports
202
203 [Container] 2026/04/27 01:17:14.028737 Running command pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report=html:reports/htmlcov --cov-report-term
204
205 test session starts
206 platform linux -- Python 3.11.15, pytest-7.4.4, pluggy-1.6.0 -- /root/.pyenv/versions/3.11.15/bin/python
207 cachedir: .pytest_cache
208 rootdir: /codebuild/output/src52826089/src/github.com/breinergonza/devops_taller_2
209 configfile: pytest.ini
210 plugins: cov-4.1.0
211 testpaths: tests
212 collecting ... collected 12 items
213
214 tests/test_blacklist_get.py::test_get_blacklist_found PASSED [ 8%]
215 tests/test_blacklist_get.py::test_get_blacklist_not_found PASSED [ 16%]
216 tests/test_blacklist_get.py::test_get_blacklist_unauthorized PASSED [ 25%]
217 tests/test_blacklist_get.py::test_get_blacklist_returns_latest_reason PASSED [ 33%]
218 tests/test_blacklist_post.py::test_post_blacklist_ok PASSED [ 41%]
219 tests/test_blacklist_post.py::test_post_blacklist_without_blocked_reason PASSED [ 50%]
220 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_without_token PASSED [ 58%]
221 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_with_bad_token PASSED [ 66%]
222 tests/test_blacklist_post.py::test_post_blacklist_missing_requires_field PASSED [ 75%]
223 tests/test_blacklist_post.py::test_post_blacklist_blocked_reason_too_long PASSED [ 83%]
224 tests/test_health.py::test_health_check_returns_200 PASSED [ 91%]
225 tests/test_health.py::test_index_returns_metadata PASSED [100%]
226
227 - generated xml file: /codebuild/output/src52826089/src/github.com/breinergonza/devops_taller_2/reports/junit.xml -
228
229 ----- coverage: platform linux, python 3.11.15-final-0 -----
230 Name Sests Miss Cover Missing
231
232 src/_init_.py 0 0 100%
233 src/main.py 51 0 88% 112-113, 151-152, 178-179
234 src/models/_init_.py 0 0 100%
235 src/models/blacklist.py 12 1 92% 22
236 src/schemas/_init_.py 0 0 100%
237 src/schemas/blacklist_schema.py 10 10 0% 2-17
238 src/views/_init_.py 0 0 100%
239 src/views/blacklist_view.py 39 2 97% 113
240
241 TOTAL 112 18 84%
242 Coverage HTML written to dir reports/htmlcov
243 Coverage XML written to file reports/coverage.xml
244
245 ===== 12 passed in 0.64s =====
246
247 [Container] 2026/04/27 01:17:16.753715 Phase complete: PRE_BUILD State: SUCCEEDED
248 [Container] 2026/04/27 01:17:16.753744 Phase context status code: Message:
```

Ilustración 5. Salida de pytest con pruebas unitarias exitosas.

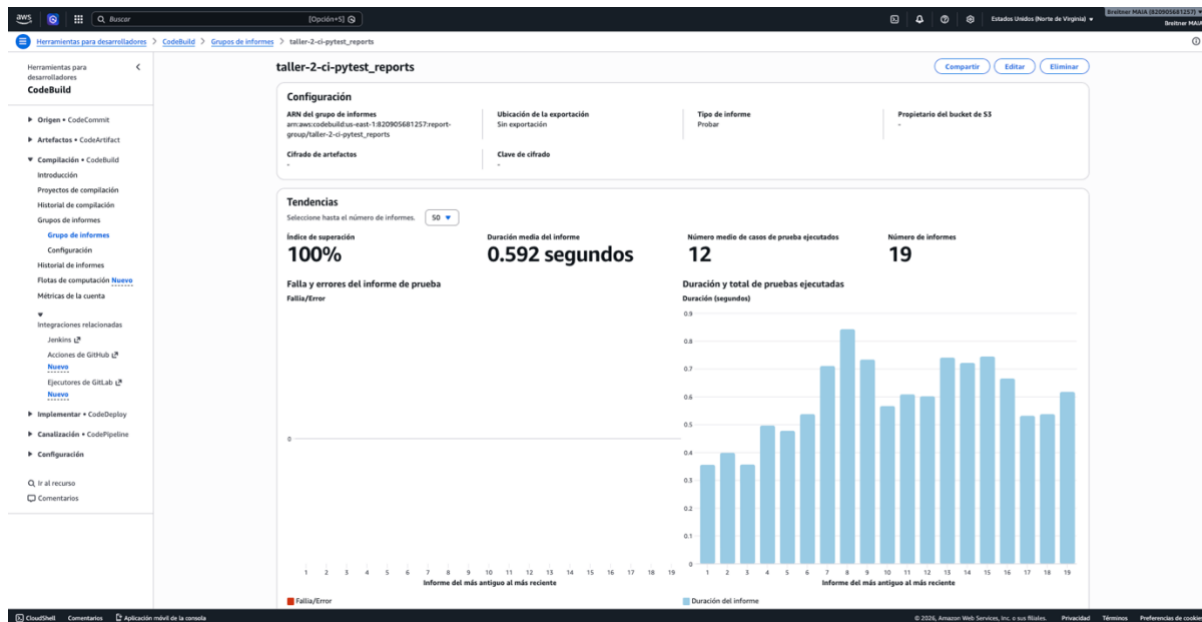


Ilustración 6. Detalle de salida de pytest con pruebas unitarias exitosas.

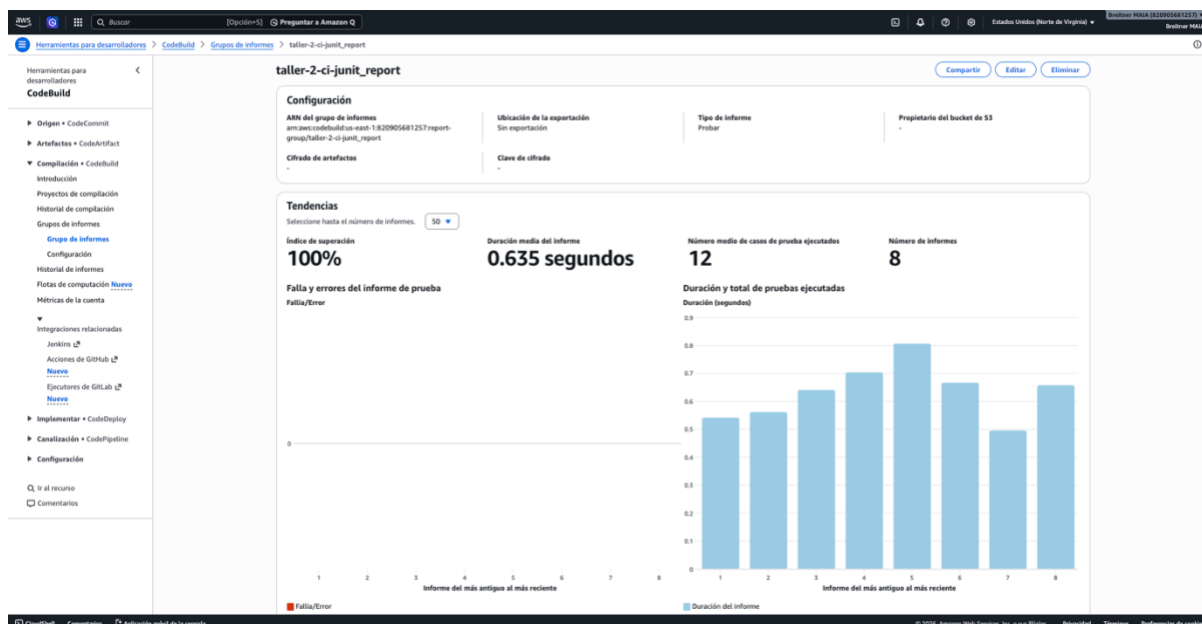


Ilustración 7. Detalle de salida de junit con pruebas unitarias exitosas.

- Framework: pytest con fixtures para cliente Flask y base de datos SQLite en memoria
- Pruebas unitarias de endpoints REST (GET /blacklists, POST /blacklists)
- Validación de autenticación con tokens Bearer y JWT
- Reportes JUnit XML para visualización en CodeBuild
- Cobertura de código con pytest-cov en formatos XML y HTML

Manejo de Fallos

```
147 tests/test_blacklist_get.py::test_get_blacklist_returns_latest_reason PASSED [ 33%]
148 tests/test_blacklist_post.py::test_post_blacklist_ok FAILED [ 41%]
149 tests/test_blacklist_post.py::test_post_blacklist_without_blocked_reason PASSED [ 50%]
150 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_without_token PASSED [ 58%]
151 tests/test_blacklist_post.py::test_post_blacklist_unauthorized_with_bad_token PASSED [ 66%]
152 tests/test_blacklist_post.py::test_post_blacklist_missing_required_field PASSED [ 75%]
153 tests/test_blacklist_post.py::test_post_blacklist_blocked_reason_too_long PASSED [ 83%]
154 tests/test_health.py::test_health_check_returns_200 PASSED [ 91%]
155 tests/test_health.py::test_index_returns_metadata PASSED [100%]

===== FAILURES =====
156 tests/test_blacklist_post.py::test_post_blacklist_ok
157 tests/test_blacklist_post.py:16: in test_post_blacklist_ok
158     assert body["email"] == "test@unandes.edu.co"
159 E   AssertionError: assert 'lol@unandes.edu.co' == 'test@unandes.edu.co'
160 E   + test@unandes.edu.co
161 E   - lol@unandes.edu.co
162 E   + test@unandes.edu.co
163 E   - lol@unandes.edu.co
164 E   + lol@unandes.edu.co
165 E   + test@unandes.edu.co
166 generated xml file: /codebuild/output/src626866186/src/github.com/breinegonza/devops_taller_2/reports/junit.xml -
167

===== coverage: platform linux, python 3.11.15-final-0 =====
168
169 Name                               Stmts   Miss  Cover   Missing
170 -----
171 src/_init_.py                       0     0   100%
172 src/main.py                         29     0   100%
173 src/models/_init_.py                0     0   100%
174 src/models/blacklist.py             12     1    92%   22
175 src/schemas/_init_.py              0     0   100%
176 src/schemas/blacklist_schema.py    10    10    0%   2-17
177 src/views/_init_.py                 0     0   100%
178 src/views/blacklist_view.py         39     1    97%   65
179 -----
180 TOTAL                               90    12    87%
181 Coverage XML written to file reports/coverage.xml
182
183 ===== short test summary info =====
184 FAILED tests/test_blacklist_post.py::test_post_blacklist_ok - AssertionError: ...
185 1 failed, 11 passed in 0.36s
186
187 [Container] 2026/04/26 03:22:38.886990 Command did not exit successfully pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report-term-missing exit status 1
188 [Container] 2026/04/26 03:22:38.891999 Phase complete: PRE_BUILD State: FAILED
189 [Container] 2026/04/26 03:22:38.892815 Phase context status code: COMMAND_EXECUTION_ERROR Message: Error while executing command: pytest -v --junitxml=reports/junit.xml --cov=src --cov-report=xml:reports/coverage.xml --cov-report-term-missing. Reason: exit status 1
190
```

Ilustración 8. Pipeline detenido cuando fallan las pruebas automáticas.

Cuando una prueba falla, CodeBuild marca el build como FAILED y CodePipeline detiene el pipeline, previniendo el despliegue de código defectuoso a producción.

- El código NO se despliega si las pruebas fallan
- Logs detallados quedan disponibles en CloudWatch
- El pipeline queda en estado Failed hasta el siguiente push exitoso

Configuración de ECS Fargate y Load Balancer

Amazon ECS Fargate ejecuta el microservicio como contenedor sin necesidad de administrar servidores. El **Application Load Balancer** distribuye el tráfico entre los ambientes Blue y Green durante los despliegues.

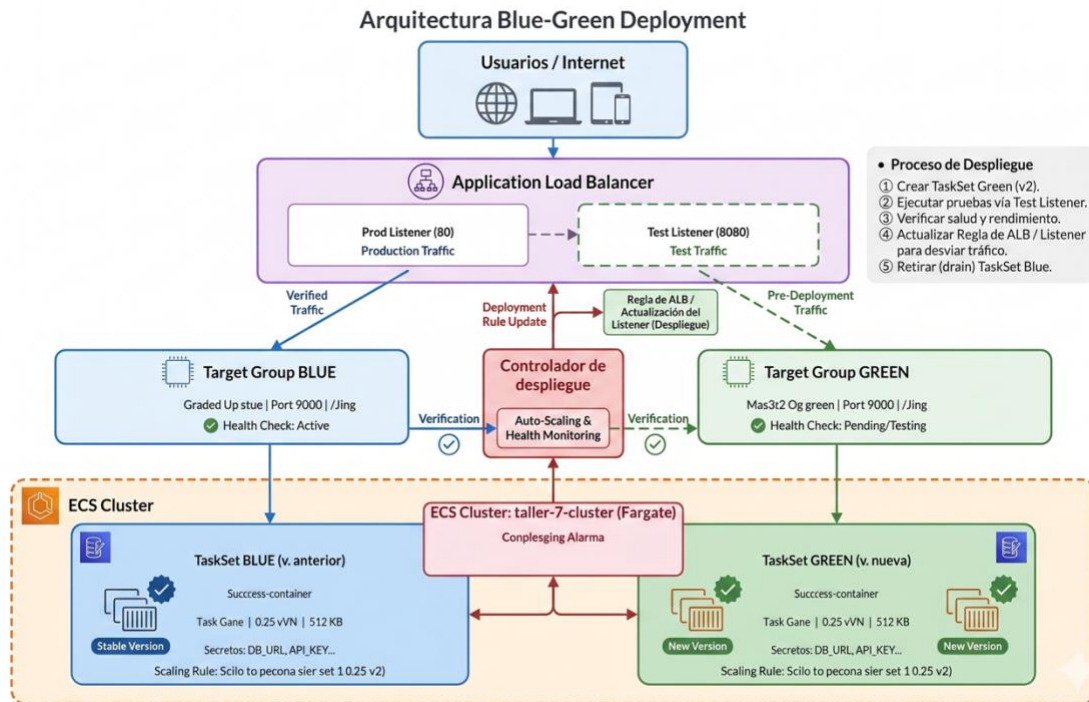


Ilustración 9. Arquitectura Blue-Green con ALB, Target Groups y ECS Fargate.

Task Definition

La Task Definition define cómo se va a ejecutar el contenedor del microservicio. Incluye la configuración de recursos, secretos y logging.

- **CPU:** 256 unidades (0.25 vCPU) / Memoria: 512 MB
- **Imagen:** Inyectada dinámicamente por CodeDeploy desde el placeholder <IMAGE1_NAME>
- **Puerto:** 5000 (TCP) mapeado al host
- **Secrets:** DATABASE_URI, JWT_SECRET_KEY, STATIC_BEARER desde Parameter Store
- **Logs:** Enviados a CloudWatch en /ecs/blacklist-service

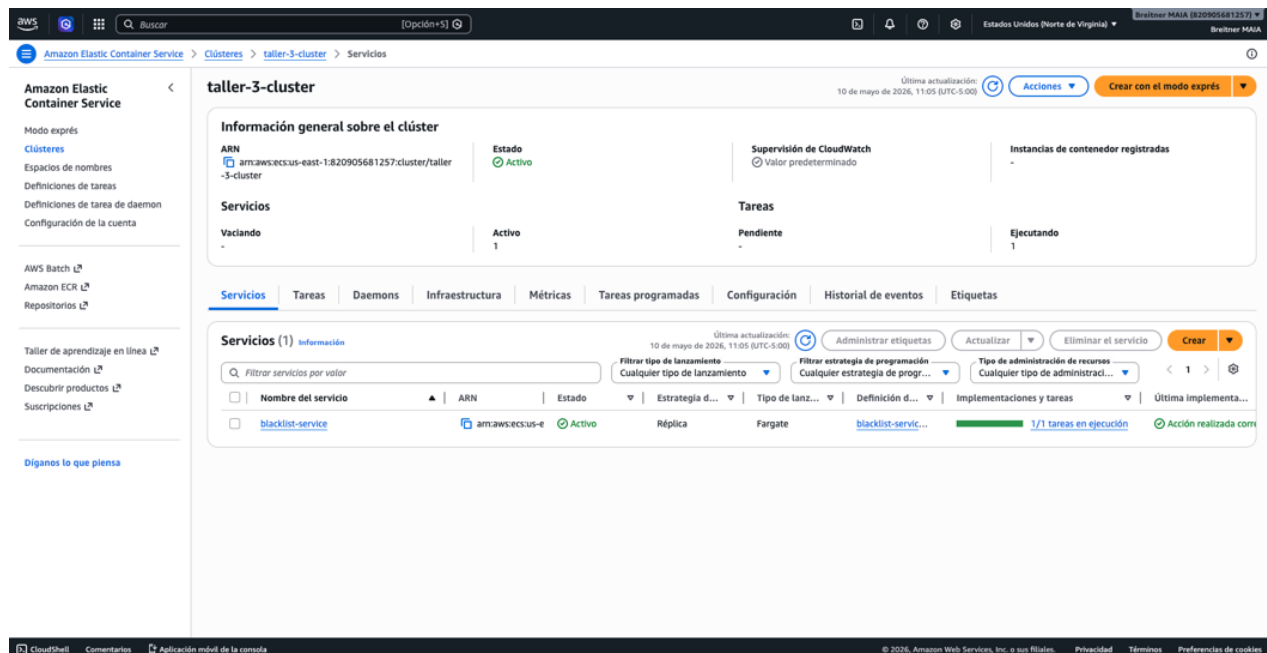


Ilustración 10. Amazon ECS - Servicio blacklist-service ACTIVE con tasks running.

Application Load Balancer

El ALB (blacklist-alb) es la pieza clave del despliegue Blue-Green. Tiene dos listeners y dos target groups que CodeDeploy alterna durante cada despliegue.

- **Listener producción (puerto 80):** Dirige tráfico real de usuarios al ambiente activo
- **Listener test (puerto 8080):** Permite validar el nuevo ambiente antes de cambiar tráfico
- **Target Group Blue (blacklist-tg-blue):** Puerto 5000, health check en /ping
- **Target Group Green (blacklist-tg-green):** Puerto 5000, health check en /ping
- **Security Group:** Permite tráfico entrante en puertos 80, 8080 y 5000

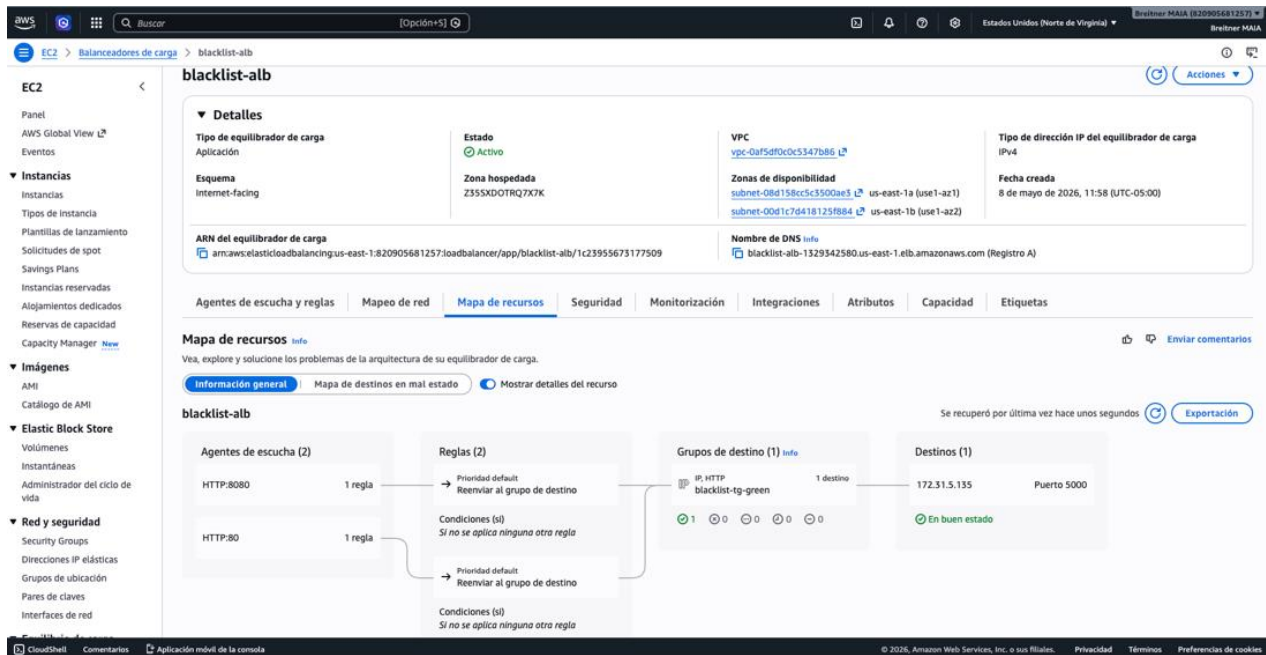


Ilustración 11. Target Groups del ALB con targets healthy.

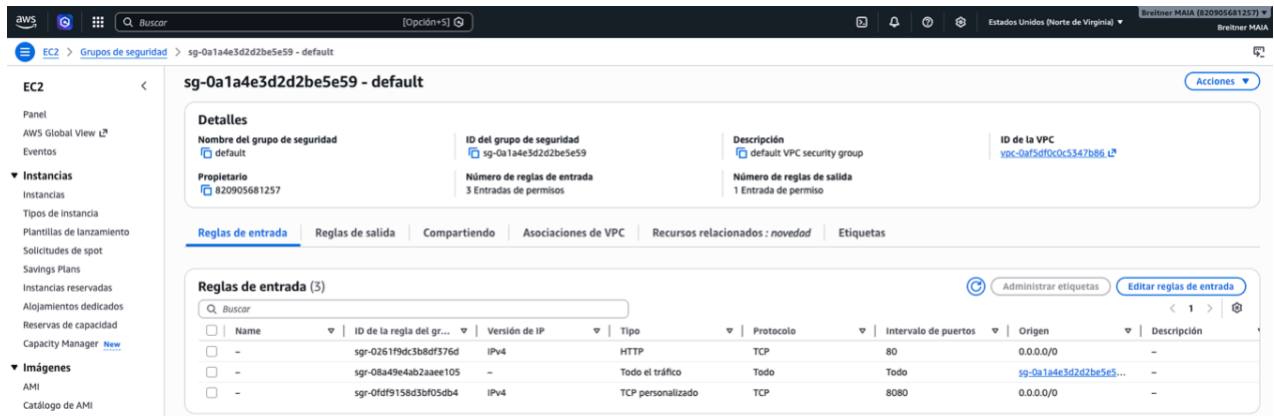


Ilustración 12. Security Group del ALB con las reglas de entrada.

Configuración de CodeDeploy Blue-Green

AWS CodeDeploy gestiona la estrategia Blue-Green sobre ECS. La aplicación "blacklist-service-app" y el grupo de despliegue "blacklist-dg" orquestan el proceso de crear el nuevo TaskSet, validar la salud y redirigir tráfico.

AppSpec y Task Definition

Los archivos appspec.yaml y task-def.json trabajan juntos para definir el despliegue:

- **appspec.yaml:** Define el servicio ECS destino con placeholder <TASK_DEFINITION> para inyección automática
- **task-def.json:** Plantilla de Task Definition con placeholder <IMAGE1_NAME> que CodeDeploy reemplaza con la URI real de ECR
- **Configuración de red:** VPC con 2 subnets, security group y IP pública habilitada

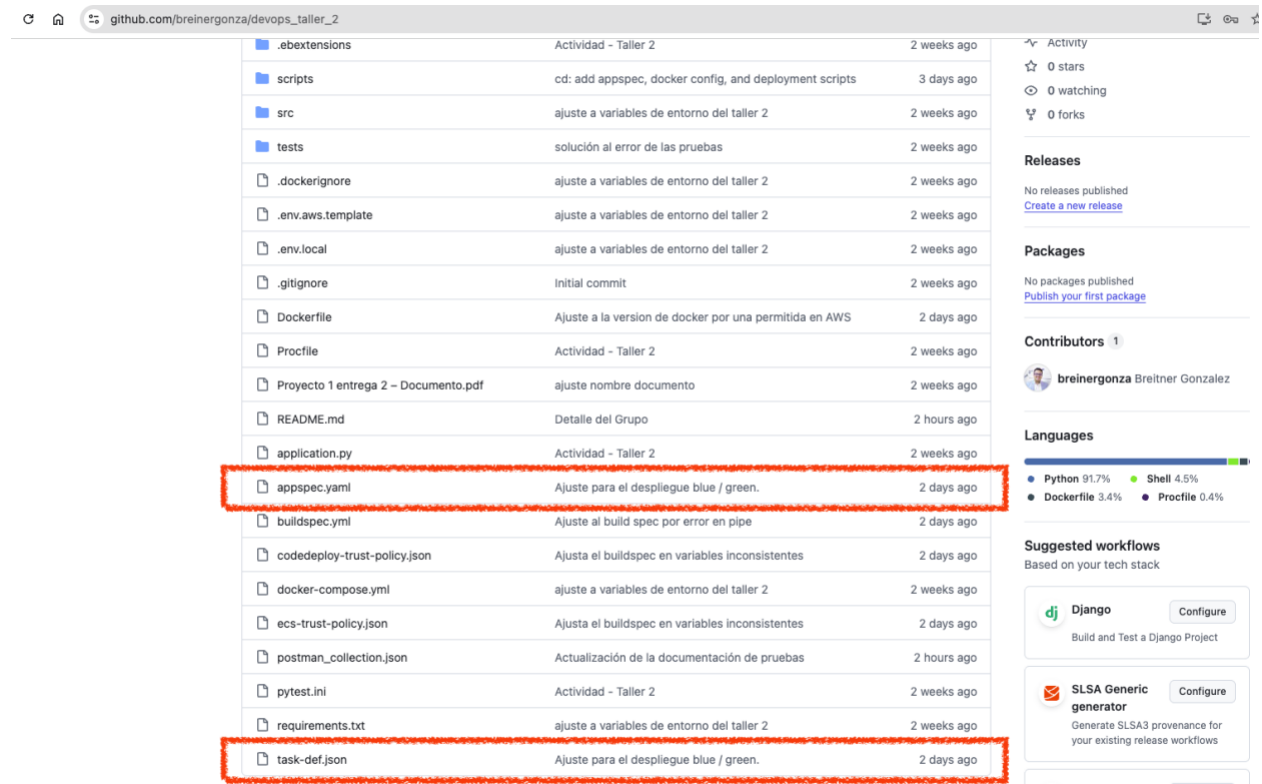


Ilustración 13. Archivos appspec.yaml y task-def.json en el repositorio.

Proceso de Despliegue Blue-Green



Ilustración 14. Proceso de despliegue Blue-Green con CodeDeploy en 6 pasos

1. CodeDeploy registra nueva Task Definition reemplazando <IMAGE1_NAME> con la URI de la imagen
2. Crea un nuevo TaskSet (Green) en el servicio ECS con la nueva Task Definition
3. El ALB health check en /ping valida que el contenedor Green está saludable
4. CodeDeploy redirige el listener de producción (puerto 80) al Target Group Green
5. El TaskSet anterior (Blue) se mantiene como respaldo por el período de espera configurado
6. Rollback automático si el health check falla o si se detecta una alarma

Historial de implementaciones

ID de la impl...	Estado	Tipo de imple...	Plataforma de L...	Aplicación	Grupo de imple...	Ubicación de la...	Iniciando evento	Hora de inicio
d-QWWPY25DJ	Realizado correctamente	Azul/verde	Amazon ECS	blacklist-service-app	blacklist-dg	5c82bf02cafee...	Acción del usuario	May. 10, 2026 9...
d-59RL4WRDJ	Realizado correctamente	Azul/verde	Amazon ECS	blacklist-service-app	blacklist-dg	41c093bed3b2e...	Acción del usuario	May. 10, 2026 9...
d-85EVAQRDJ	Realizado correctamente	Azul/verde	Amazon ECS	blacklist-service-app	blacklist-dg	1b4d0f9749d9f...	Acción del usuario	May. 10, 2026 9...
d-9M836ARDJ	Realizado correctamente	Azul/verde	Amazon ECS	blacklist-service-app	blacklist-dg	73f14803273e8...	Acción del usuario	May. 10, 2026 8...
d-03ZQ5ZCJ	Realizado correctamente	Azul/verde	Amazon ECS	blacklist-service-app	blacklist-dg	3b9efa371derf...	Acción del usuario	May. 8, 2026 12...

Ilustración 15. CodeDeploy – Historial de despliegues Blue-Green exitoso con alternancia de tráfico.

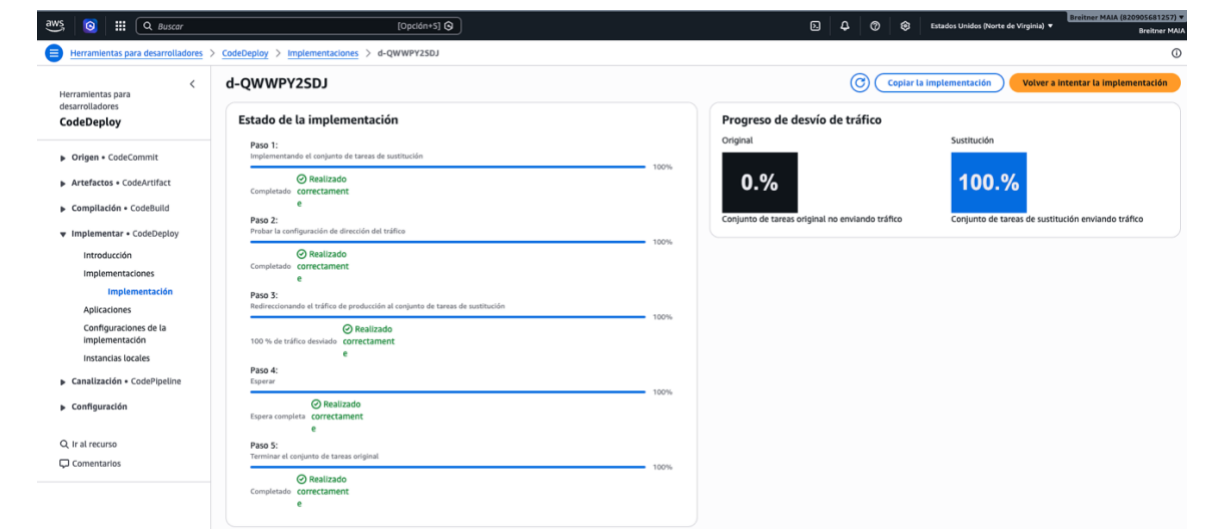


Ilustración 16. CodeDeploy – Estados despliegue Blue-Green exitoso con alternancia de tráfico.

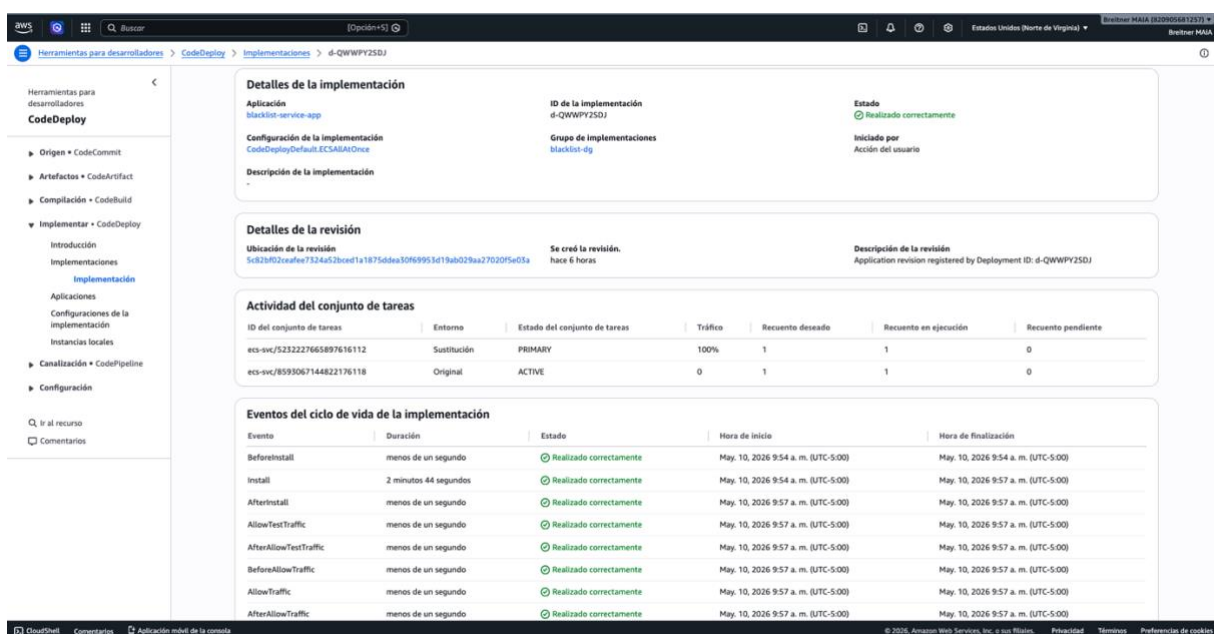


Ilustración 17. CodeDeploy – Detalle despliegue Blue-Green exitoso con alternancia de tráfico.

Pipeline Completo en CodePipeline

AWS CodePipeline V2 orquesta las tres etapas del pipeline y se dispara automáticamente mediante webhook de GitHub cada vez que hay un push a la rama main.

Configuración del Pipeline

Nombre: taller-3-pipeline (tipo V2 con modo QUEUED)

Source: Código fuente en GitHub ([breinergonza/devops_taller_2](https://github.com/breinergonza/devops_taller_2), rama **main**)

Build: Proyecto CodeBuild taller-2-ci con privilegedMode para Docker-in-Docker

Deploy: Despliegue con Application blacklist-service-app y DeploymentGroup blacklist-dg

Trigger: Webhook automático que detecta pushes a la rama main

Webhook en el Repositorio en Github

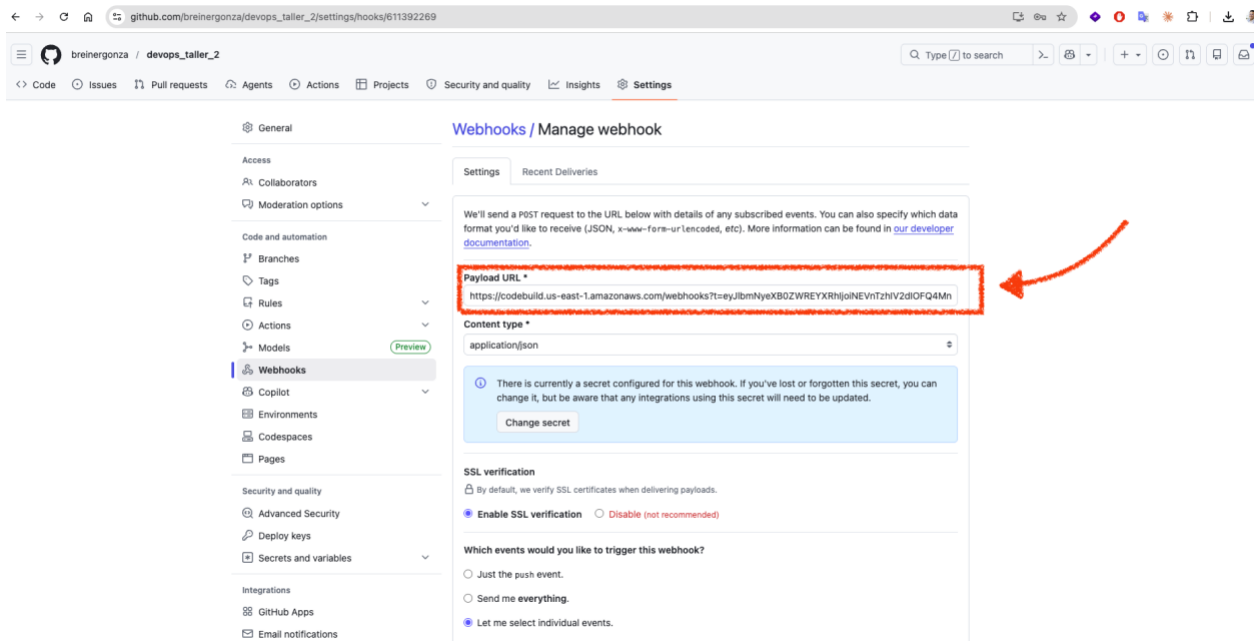


Ilustración 18. Webhook que detecta pushes en Github.

Ejecución Exitosa del Pipeline

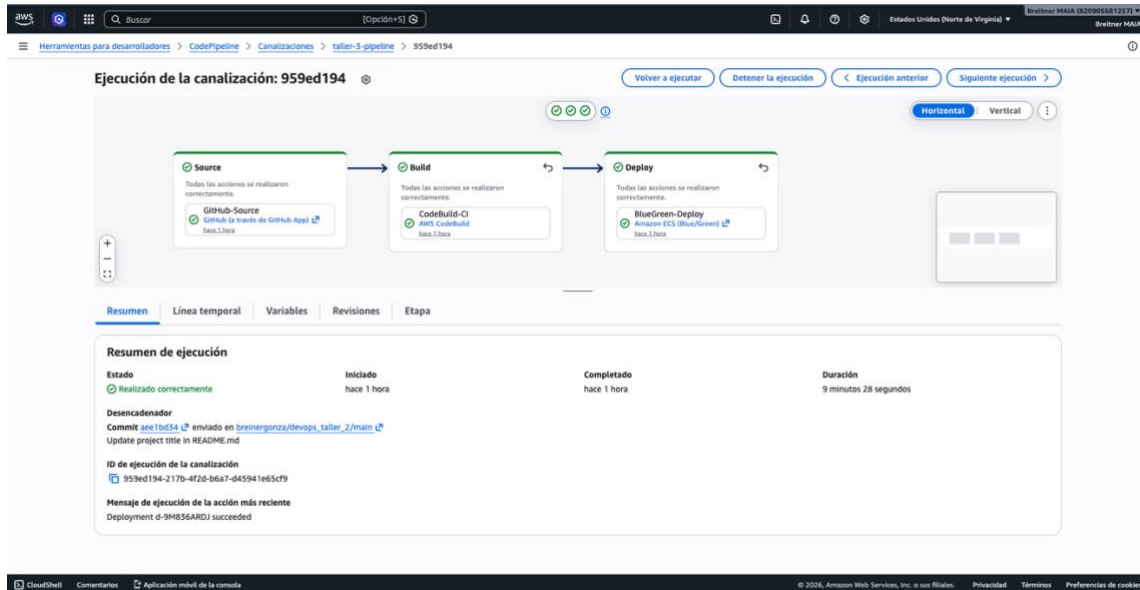


Ilustración 19. Pipeline exitoso - Las 3 etapas Source, Build y Deploy en estado Succeeded.

El pipeline se ejecutó exitosamente de punta a punta. Las tres etapas (Source, Build, Deploy) completaron sin errores, desplegando automáticamente la nueva versión del microservicio con estrategia Blue-Green.

1. **Source:** Descarga del código fuente desde GitHub (Succeeded)
2. **Build:** Pruebas unitarias, construcción Docker, push a ECR (Succeeded)
3. **Deploy:** Blue-Green deployment en ECS vía CodeDeploy (Succeeded)

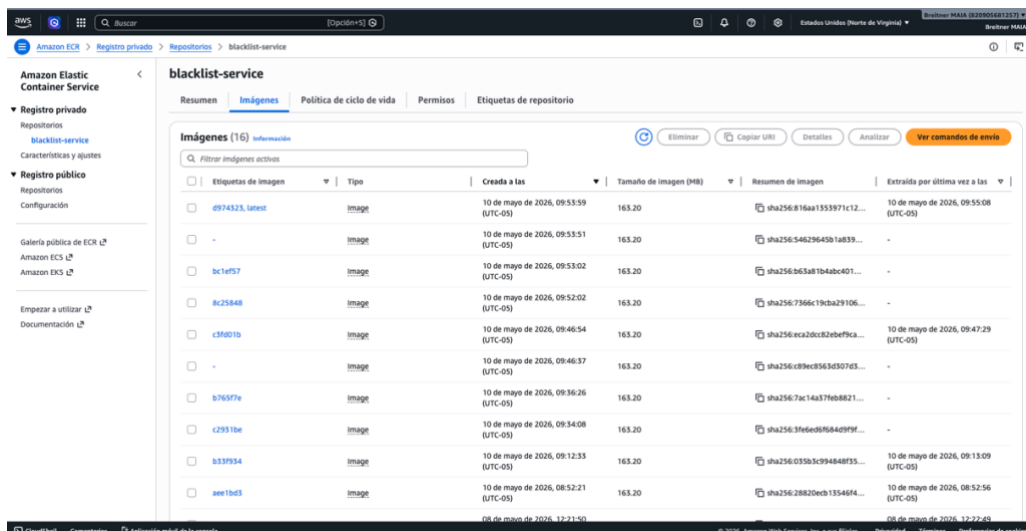


Ilustración 20. Amazon ECR - Repositorio blacklist-service con imágenes Docker publicadas.

Validación del Despliegue

Después del despliegue Blue-Green exitoso, se validó que el microservicio responde correctamente a través del Application Load Balancer.

- **Endpoint de salud:** GET /ping → {"status": "ok", "timestamp": "..."}
- **Target Group Green:** 1 target healthy en puerto 5000
- **ALB DNS:** blacklist-alb-1329342580.us-east-1.elb.amazonaws.com
- **Servicio ECS:** ACTIVE con 1/1 tasks running

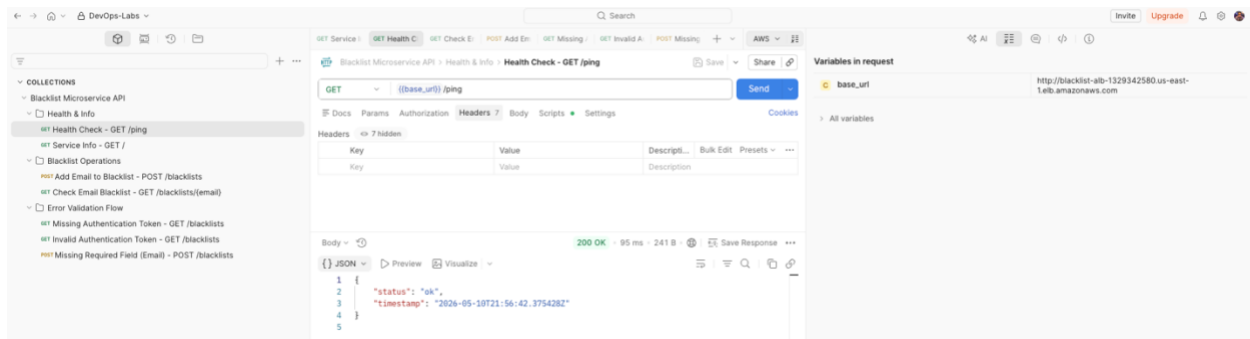


Ilustración 21. Prueba del despliegue con Postman.

Gestión de Variables de Entorno

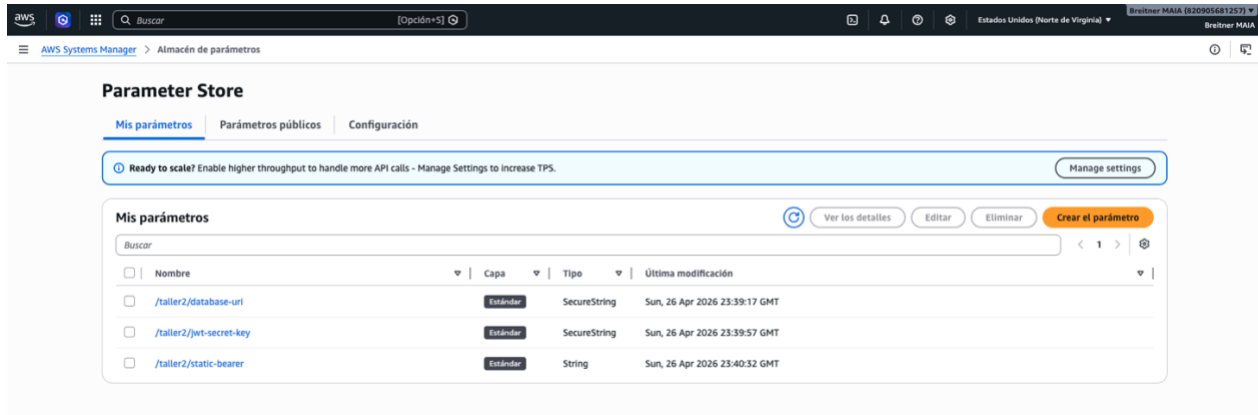


Ilustración 13. AWS Systems Manager - Parámetros SecureString configurados.

AWS Systems Manager Parameter Store almacena los secretos de forma centralizada y segura como SecureString. Los secretos se inyectan automáticamente en el contenedor ECS a través de la Task Definition, sin exponerlos en el código fuente ni en los artefactos del build.

- **DATABASE_URI:** Cadena de conexión a la base de datos PostgreSQL en RDS (/taller2/database-uri)
- **JWT_SECRET_KEY:** Clave secreta para generación y validación de tokens JWT (/taller2/jwt-secret-key)
- **STATIC_BEARER:** Token de acceso estático para autenticación de servicios (/taller2/static-bearer)

Conclusiones

En este taller se muestra la implementación exitosa de un pipeline de Entrega Continua completo usando servicios de AWS. El pipeline automatiza el flujo desde un push en GitHub hasta un despliegue Blue-Green en producción, garantizando cero downtime y la capacidad de rollback instantáneo ante cualquier problema.

La estrategia Blue-Green con CodeDeploy demostró ser la pieza más valiosa de la arquitectura. Al mantener dos ambientes y alternar tráfico a través del ALB, cada despliegue es seguro y reversible. Los health checks automáticos en el endpoint /ping aseguran que el nuevo ambiente funciona antes de recibir tráfico real.

Los desafíos que encuentre durante la implementación de este ejercicio giraron entorno a los “rate limiting” de Docker Hub, a formatos de artefactos diferentes entre ECS rolling update y CodeDeploy, y algunos en la configuración de los security groups, los cuales fueron un poquito complicados de resolver, sin embargo este tipo de prácticas son super importantes ya que no llevan a tener que entender en profundidad cada servicio de AWS y cómo estos interactúan entre sí.

El resultado final es un pipeline robusto que refleja las mejores prácticas de la industria: integración continua con pruebas automatizadas, contenedorización con Docker, registro de imágenes en ECR, despliegue Blue-Green con CodeDeploy, y gestión segura de secretos con Parameter Store. Este pipeline puede servir como base para cualquier microservicio que requiera entregas frecuentes y seguras a producción.

Referencias

AWS. (2026). Documentación de AWS CodePipeline. Recuperado de <https://docs.aws.amazon.com/codepipeline/>

AWS. (2026). Documentación de AWS CodeDeploy. Recuperado de <https://docs.aws.amazon.com/codedeploy/>

AWS. (2026). Documentación de Amazon ECS. Recuperado de <https://docs.aws.amazon.com/ecs/>

AWS. (2026). Documentación de AWS CodeBuild. Recuperado de <https://docs.aws.amazon.com/codebuild/>

AWS. (2026). Blue/Green Deployments with CodeDeploy. Recuperado de <https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>

AWS. (2026). Documentación de Elastic Load Balancing. Recuperado de <https://docs.aws.amazon.com/elasticloadbalancing/>